

Air University



COAL (Lab)

Year 2025

Ms. Sonia Shehzadi

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: December 26, 2025

Lab Task # 15

Part 1: Inline Assembly in Dev C++

Write a C++ program that uses inline assembly to add two integers and returns the result.

Details:

- Write a C++ function `int add (int a, int b)` which uses Dev C++ inline assembly to perform the addition.
- Call this function from `main ()` and print the result.

Code

```
#include <iostream>
using namespace std;

int add(int a, int b) // Inline assembly function
{
    int result;

    asm volatile(
        "movl %1, %%eax;"
        "addl %2, %%eax;"
        "movl %%eax, %0;"
        : "=r"(result)
        : "r"(a), "r"(b)
        : "%eax");

    return result;
}

int main()
{
    int x = 5, y = 7;
    cout << "\nSum = " << add(x, y) << endl;
    return 0;
}
```

Explanation:

This code demonstrates how to embed assembly instructions directly within a C++ function using the GCC/Dev-C++ compiler.

1. **Extended asm Block:** The *asm volatile* block allows you to insert raw assembly instructions into C++. The *volatile* keyword tells the compiler not to "optimize away" or change the order of these instructions, ensuring they run exactly as written.

2. **Input and Output Operands:** The syntax uses placeholders like `%0`, `%1`, and `%2`. These map to the C++ variables listed at the bottom of the block: `%0` is the *result* (output), while `%1` and `%2` are the inputs `a` and `b`.
3. **Register Management:** The instruction `movl %1, %%eax` moves the value of variable `a` into the EAX register. The double percent signs (`%%eax`) are required in this syntax to distinguish CPU registers from the numbered C++ placeholders.
4. **The Addition Logic:** `addl %2, %%eax` adds the value of variable `b` to the current value in the EAX register. The result of the addition is then moved from the register into the C++ variable `result` using the instruction `movl %%eax, %0`.
5. **Clobber List:** The final part of the block ("`%eax`") is called the "clobber list." It warns the C++ compiler that the assembly code is using the EAX register, so the compiler should avoid using that register for other tasks at the same time to prevent data corruption.

Output



Part 2: External Linkage with C++ using MSVC

Write a C++ program that calls an external assembly function to add two integers.

- Write the assembly function `int add (int a, int b)` in a separate assembly file (`add.asm`).
- The assembly function should take two integers as input and return their sum.
- Write a C++ program (`main.cpp`) that declares this function as `extern "C"` and calls it.
- Display the result in the C++ program.

Code

`add.asm`

```
; add.asm (64-bit)
global add

section .text
add:
    mov rax, rcx    ; rax = first parameter
    add rax, rdx    ; rax = rax + second parameter
    ret
```

Explanation:

This code is a **64-bit Assembly function** designed to be called from a C++ program. It follows the **x64 Calling Convention** (specifically the Microsoft x64 convention).

1. **x64 Calling Convention:** In 64-bit Windows systems, the first two parameters of a function are automatically passed into the **RCX** and **RDX** registers. This is more efficient than 32-bit systems which use the stack.
2. **64-Bit Arithmetic:** The code uses **RAX**, **RCX**, and **RDX**, which are the 64-bit versions of the older EAX, ECX, and EDX registers. This allows the function to process much larger numbers (64-bit integers).
3. **Return Register:** In assembly-to-C++ interfacing, the **RAX** register is the designated return register. When `ret` is called, C++ automatically looks at the value in **RAX** and uses it as the function's result.

main.cpp

```
#include <iostream>
using namespace std;

extern "C" int add(int a, int b);

int main()
{
    int a = 10, b = 20;
    cout << "\nSum = " << add(a, b) << endl;
    return 0;
}
```

Explanation:

This C++ code demonstrates **External Linkage**, which is Part 2 of your Lab Task. It allows a C++ file to communicate with a completely separate Assembly file.

Here is the explanation in 4 points:

1. **The extern "C" Declaration:** This is the most important line. It tells the C++ compiler that the function `add` is defined in an external file and should use the "C" naming convention. Without this, C++ would "mangle" the function name, making it impossible to link with the Assembly code.
2. **Function Prototype:** By declaring `int add (int a, int b)`, you are telling the compiler the signature of the function (it takes two integers and returns one). This allows the C++ compiler to set up the registers or stack correctly before jumping to the Assembly code.

3. **Linking Process:** This code does not contain the logic for addition. Instead, during the "linking" phase of compilation, the computer searches for an object file (compiled from your .asm file) that contains a label named add and joins it with this C++ program.
4. **Abstraction:** This approach demonstrates a professional workflow where high-level logic (like printing with cout) is handled by C++, while the performance-critical or low-level math logic is handled by Assembly, keeping the two concerns separate.

Output



```
Sum = 30
```

THE END
